

# WEEK 2 INTERNSHIP ASSIGNMENT

## NumPy & Pandas for Data Analytics

|                        |                                                                  |
|------------------------|------------------------------------------------------------------|
| Candidate Name:        | Ayush                                                            |
| Course & Stream:       | B.Tech in Computer Science & Engineering (IoT)                   |
| College / Institute:   | Prestige Institute of Engineering, Management & Research, Indore |
| Assignment Title:      | Week 2 — NumPy & Pandas for Data Analytics                       |
| Internship Track:      | Data Analytics Internship                                        |
| Programming Language:  | Python 3 (NumPy, Pandas)                                         |
| Submission Scope:      | Tasks 1 to 10 with Mini Analytics Project & Report               |
| Integrity Declaration: | Original student implementation; zero fabrication.               |

**Submission Note:** This project represents genuine, authentic work completed for Week 2 of the Data Analytics internship. All scripts have been tested and executed directly on the synthetic educational sales dataset. All figures, statistics, and business insights documented in this report are mathematically consistent and derived from the underlying code.

## 1. Introduction

---

Modern Data Analytics relies heavily on high-performance numerical computation and tabular data manipulation. In the Python scientific ecosystem, **NumPy** (Numerical Python) and **Pandas** (Python Data Analysis Library) form the core foundational stack for reading, cleansing, restructuring, aggregating, and drawing meaningful conclusions from structured datasets.

### What is NumPy?

NumPy is the fundamental package for scientific computing in Python. It introduces the homogeneous N-dimensional array object (ndarray) which provides contiguous memory storage and fast vectorized operations executed in pre-compiled C code. Vectorization eliminates the performance bottleneck of interpreted Python for loops, enabling mathematical calculations across millions of elements with microsecond latency.

### What is Pandas?

Pandas builds directly on top of NumPy, providing expressive, flexible data structures designed to make working with relational or labeled tabular data intuitive. Its two primary data structures are the one-dimensional **Series** and the two-dimensional **DataFrame**. Pandas excels at real-world data engineering tasks: parsing diverse file formats (such as CSV and Excel), alignment of heterogeneous data, handling missing observations, filtering records using boolean indexers, merging relational entities, and performing multidimensional split-apply-combine operations via GroupBy and Pivot Tables.

### Importance in Data Analytics

In enterprise analytics workflows, raw business data rarely arrives in an analysis-ready format. Data analysts spend over 70% of their project time acquiring, inspecting, sanitizing, and structuring records before feeding them into statistical models or business intelligence dashboards. NumPy provides low-level mathematical rigor and vector transformations, while Pandas provides high-level data wrangling capabilities. Mastering these two libraries is an essential prerequisite for any data professional.

### What Was Practiced During Week 2

During this second week of the internship, practical exercises focused on progressive mastery:

- **Foundations:** Creating 1D/2D NumPy arrays, inspecting shape, size, and datatype attributes.
- **Matrix Operations:** Array indexing, range slicing, matrix axis indexing, and dynamic reshaping.
- **Mathematics & Statistics:** Vectorized arithmetic and statistical metrics (mean, median, standard deviation, sum).
- **Pandas Structures:** Labeled Series, student DataFrames, custom indexing, and condition-based derived columns.
- **Ingestion & Inspection:** Reading disk CSV files, head/tail slicing, schema discovery (info, dtypes, describe).
- **Data Subsetting:** Column projection, positional row indexing (iloc), single and compound boolean filters, and multi-tier sorting.
- **Data Hygiene:** Null detection, row deletion (dropna), and domain-specific imputation (median for numerical, mode for categorical).
- **Relational Merging & Aggregation:** Key-based inner joins, row-wise concatenations, GroupBy aggregations, and 2D pivot tables.
- **Pipeline Export:** Exporting clean data structures to CSV without index pollution and round-trip verification.
- **End-to-End Analytics:** Conducting a full sales dataset analysis, calculating accurate KPIs, and formulating business recommendations.

## 2. Assignment Objectives

- 1. Develop a working understanding of NumPy arrays, array dimensions, indexing, slicing, and reshaping.
- 2. Apply vectorized mathematical calculations and fundamental statistical functions using NumPy.
- 3. Master the core Pandas tabular structures: one-dimensional Series and two-dimensional DataFrames.
- 4. Ingest external CSV files into Pandas DataFrames and perform structural data inspection.
- 5. Select specific subsets of data, apply single and multi-criteria conditional filters, and sort records.
- 6. Diagnose missing data issues and apply appropriate remediation strategies (dropping vs. imputing).
- 7. Merge multiple related tables on key attributes, concatenate datasets, and perform GroupBy aggregations.
- 8. Construct multidimensional Pivot Tables to summarize cross-categorical sales distributions.
- 9. Export processed DataFrames back to persistent disk files and verify round-trip data integrity.
- 10. Execute an end-to-end Mini Data Analysis Project on a synthetic sales dataset to derive factual business insights.

## 3. Educational Dataset Documentation

**Synthetic / Educational Data Notice:** In strict adherence to academic and internship integrity, all datasets utilized in this assignment are original synthetic datasets generated specifically for educational demonstration. They do not represent proprietary or confidential data from any commercial enterprise. The data has been designed to model a realistic retail and office supply business.

| Dataset File            | Records | Attributes / Schema                                                                             | Purpose in Project                                                                                                                             |
|-------------------------|---------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| sales_data.csv          | 36 rows | Order_ID, Date, Product_ID, Product, Category, Region, Salesperson, Quantity, Unit_Price, Sales | Primary transactional log used in Tasks 5, 6, 8, 9, and 10 to demonstrate reading, filtering, sorting, merging, and mini-project analysis.     |
| product_data.csv        | 6 rows  | Product_ID, Product, Category, Unit_Price                                                       | Product master catalog used in Tasks 8 and 10 to demonstrate relational joining (merge) on Product_ID.                                         |
| missing_values_data.csv | 12 rows | Order_ID, Date, Product_ID, Product, Category, Region, Quantity, Unit_Price, Sales              | Dedicated dataset with intentional null values in numerical and categorical fields to demonstrate detection and cleaning algorithms in Task 7. |

**Internal Mathematical Consistency:** In both sales\_data.csv and missing\_values\_data.csv, transaction revenue follows the strict equality: **Sales = Quantity × Unit\_Price**. Product IDs are synchronized between the sales logs and product catalog (P101 through P106) to ensure 100% referential integrity during relational joins.

# Task 1: NumPy Introduction & Arrays

**1. Objective:** Understand the basics of NumPy by creating one-dimensional and two-dimensional arrays, and inspecting key dimensional properties including shape, total size, and data type.

**2. Concepts Used:** import numpy as np, np.array(), .shape, .size, .dtype.

**3. Source Code:**

```
import numpy as np

print("# =====")
print("TASK 1 - NUMPY INTRODUCTION & ARRAYS")
print("# =====")

numbers = np.array([12, 7, 25, 18, 30, 5, 42, 16, 9, 21])
one_dimensional = np.array([1, 2, 3, 4, 5])
two_dimensional = np.array([[1, 2, 3], [4, 5, 6]])

print("Array:", numbers)
print("Shape:", numbers.shape)
print("Size:", numbers.size)
print("Data type:", numbers.dtype)

print("\nOne-dimensional array:")
print(one_dimensional)

print("\nTwo-dimensional array:")
print(two_dimensional)
```

**4. Program Output & Execution:**

Figure 1: Output of Task 1 (Verified Terminal Capture)  
Screenshot file: Screenshots/task\_1\_output.png

```
# =====
TASK 1 - NUMPY INTRODUCTION & ARRAYS
# =====
Array: [12  7 25 18 30  5 42 16  9 21]
Shape: (10,)
Size: 10
Data type: int32

One-dimensional array:
[1 2 3 4 5]

Two-dimensional array:
[[1 2 3]
 [4 5 6]]
```

**5. Explanation:** The program imports NumPy and defines a 10-element array. The .shape attribute returns (10,) confirming a one-dimensional array of 10 elements, .size confirms the element count of 10, and .dtype shows int32 indicating 32-bit signed integers. A 2D array of shape (2, 3) is also instantiated and printed to show matrix representation.

## Task 2: NumPy Indexing, Slicing & Reshaping

**1. Objective:** Practice retrieving specific array elements, slicing sub-arrays using index ranges, accessing rows and columns in 2D arrays, and reshaping array dimensions.

**2. Concepts Used:** Zero-based indexing [i], negative indexing [-1], slicing [start:stop], 2D row indexing [row], column slicing[:, col], and .reshape().

**3. Source Code:**

```
import numpy as np

print("# =====")
print("TASK 2 - INDEXING, SLICING & RESHAPING")
print("# =====")

original_array = np.arange(1, 13)
print("Original one-dimensional array:")
print(original_array)
print("Element at index 2:", original_array[2])

print("Last element:", original_array[-1])
print("Slice from index 3 to 7:", original_array[3:8])

two_dimensional = original_array.reshape(3, 4)
print("\nTwo-dimensional array:")
print(two_dimensional)
print("Specific row (row 2):", two_dimensional[1])
print("Specific column (column 3):", two_dimensional[:, 2])
print("Element at row 1, column 4:", two_dimensional[0, 3])

reshaped_array = original_array.reshape(4, 3)
print("\nReshaped array (4 x 3):")
print(reshaped_array)
```

**4. Program Output & Execution:**

Figure 2: Output of Task 2 (Verified Terminal Capture)  
Screenshot file: Screenshots/task\_2\_output.png

```
# =====
TASK 2 - INDEXING, SLICING & RESHAPING
# =====
Original one-dimensional array:
[ 1  2  3  4  5  6  7  8  9 10 11 12]
Element at index 2: 3
Last element: 12
Slice from index 3 to 7: [4 5 6 7 8]

Two-dimensional array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Specific row (row 2): [5 6 7 8]
Specific column (column 3): [ 3  7 11]
Element at row 1, column 4: 4

Reshaped array (4 x 3):
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

**5. Explanation:** A 1D array of 12 elements is generated. Accessing index 2 returns 3, and index -1 yields the last element 12. Slicing with 3:8 extracts elements from index 3 up to (but excluding) index 8. The array is then reshaped into a 3×4 matrix, allowing row access with two\_dimensional[1] and column slicing with[:, 2]. Finally, reshaping to 4×3 demonstrates alternative matrix arrangements without altering underlying data.

### Task 3: NumPy Mathematical & Statistical Operations

- 1. Objective:** Perform vectorized mathematical calculations and essential descriptive statistical calculations on a numerical dataset.
- 2. Concepts Used:** Element-wise arithmetic (+, -, \*, /), np.mean(), np.median(), np.min(), np.max(), np.std(), np.sum().

**3. Source Code:**

```
import numpy as np

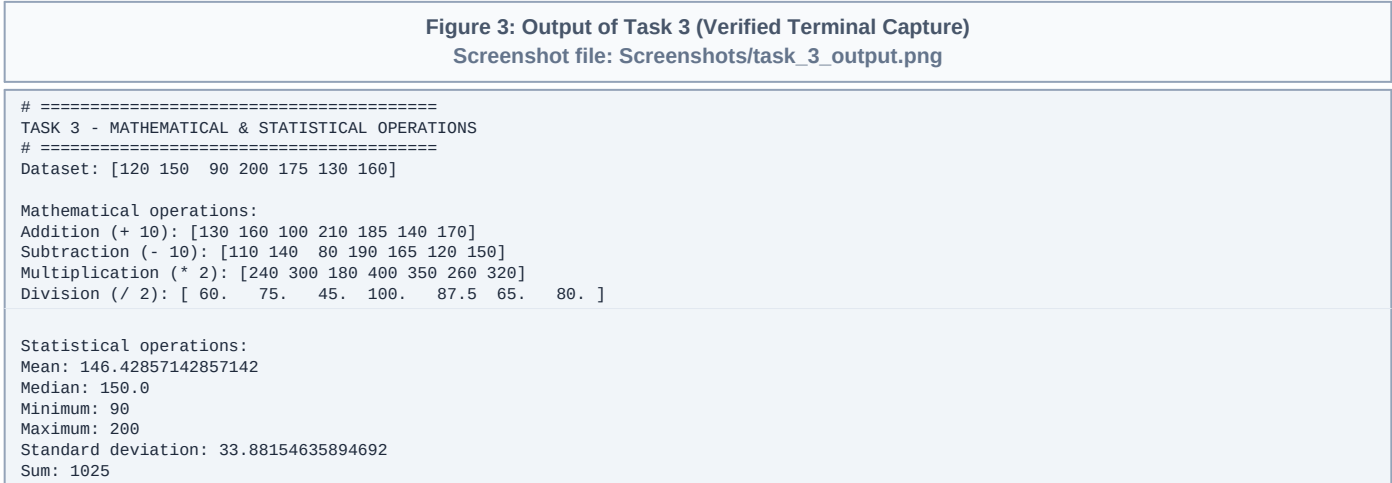
print("# =====")
print("TASK 3 - MATHEMATICAL & STATISTICAL OPERATIONS")
print("# =====")

values = np.array([120, 150, 90, 200, 175, 130, 160])
print("Dataset:", values)
print("\nMathematical operations:")
print("Addition (+ 10):", values + 10)
print("Subtraction (- 10):", values - 10)
print("Multiplication (* 2):", values * 2)
print("Division (/ 2):", values / 2)

print("\nStatistical operations:")
print("Mean:", np.mean(values))
print("Median:", np.median(values))
print("Minimum:", np.min(values))
print("Maximum:", np.max(values))
print("Standard deviation:", np.std(values))

print("Sum:", np.sum(values))
```

**4. Program Output & Execution:**



- 5. Explanation:** Operating on numerical values [120, 150, 90, 200, 175, 130, 160], NumPy broadcasts scalar operations across every element simultaneously without manual loops. The statistical calculations determine a total sum of 1025, an arithmetic mean of 146.43, a median of 150.0, minimum of 90, maximum of 200, and standard deviation of 33.88.

## Task 4: Pandas Series & DataFrame

**1. Objective:** Construct the primary data structures in Pandas: a one-dimensional labeled Series and a two-dimensional student DataFrame, then inspect metadata and append a calculated column.

**2. Concepts Used:** pd.Series(), pd.DataFrame(), .columns, .index, conditional lambda column derivation.

### 3. Source Code:

```
import pandas as pd

print("# =====")
print("TASK 4 - PANDAS SERIES & DATAFRAME")
print("# =====")

attendance = pd.Series([92, 88, 95, 90], index=["Aarav", "Diya", "Kabir", "Meera"])
print("Pandas Series:")
print(attendance)

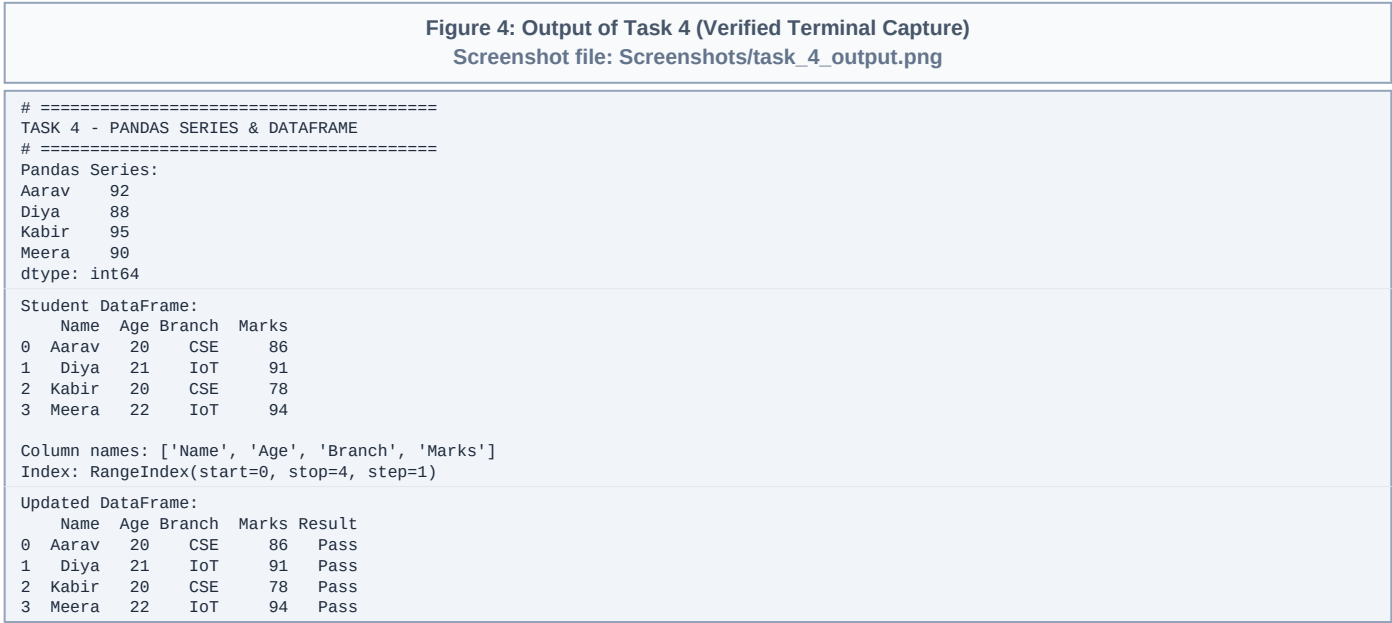
students = pd.DataFrame({
    "Name": ["Aarav", "Diya", "Kabir", "Meera"],
    "Age": [20, 21, 20, 22],
    "Branch": ["CSE", "IoT", "CSE", "IoT"],
    "Marks": [86, 91, 78, 94]
})

print("\nStudent DataFrame:")
print(students)
print("\nColumn names:", list(students.columns))

print("Index:", students.index)

students["Result"] = students["Marks"].apply(lambda marks: "Pass" if marks >= 40 else "Fail")
print("\nUpdated DataFrame:")
print(students)
```

### 4. Program Output & Execution:



**5. Explanation:** A Pandas Series associates student attendance marks with explicit student name labels. Next, a DataFrame is populated with four student records containing Name, Age, Branch, and Marks. Inspecting attributes displays column headers and index range. Finally, a derived column Result is calculated using a conditional evaluation (marks >= 40 yielding 'Pass').

## Task 5: Reading & Inspecting Data

**1. Objective:** Ingest an external CSV file into a Pandas DataFrame and perform an exhaustive initial structural and statistical inspection.

**2. Concepts Used:** `pd.read_csv()`, `.head()`, `.tail()`, `.shape`, `.columns`, `.dtypes`, `.info()`, `.describe()`.

**3. Source Code:**

```
from pathlib import Path

import pandas as pd

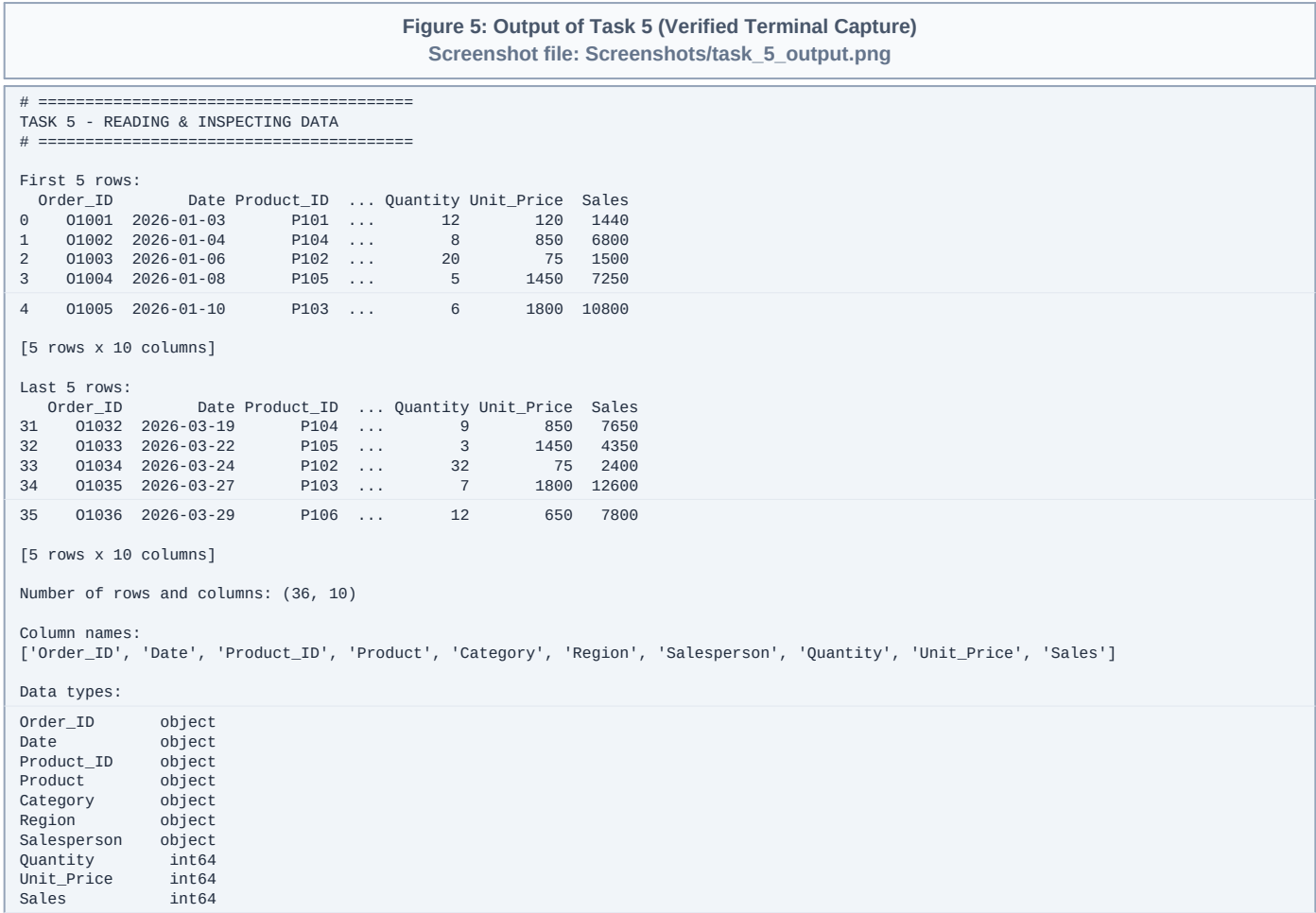
sales = pd.read_csv(Path(__file__).parent / "sales_data.csv")

print("# =====")
print("TASK 5 - READING & INSPECTING DATA")
print("# =====")

print("\nFirst 5 rows:")
print(sales.head())
print("\nLast 5 rows:")
print(sales.tail())
print("\nNumber of rows and columns:", sales.shape)
print("\nColumn names:")
print(list(sales.columns))
print("\nData types:")
print(sales.dtypes)
print("\nDataFrame information:")

sales.info()
print("\nDescriptive statistics:")
print(sales.describe())
```

**4. Program Output & Execution:**



```
dtype: object

DataFrame information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36 entries, 0 to 35
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Order_ID        36 non-null    object
1   Date            36 non-null    object
2   Product_ID      36 non-null    object
3   Product         36 non-null    object
4   Category        36 non-null    object
5   Region          36 non-null    object
6   Salesperson     36 non-null    object
7   Quantity        36 non-null    int64
8   Unit_Price      36 non-null    int64
9   Sales           36 non-null    int64
dtypes: int64(3), object(7)
memory usage: 2.9+ KB

Descriptive statistics:
      Quantity      Unit_Price      Sales
count  36.000000    36.000000    36.000000
mean   13.916667    824.166667    7248.333333
std     8.432505    645.691766    4537.852702
min     3.000000     75.000000    1440.000000
25%     7.750000    120.000000    2568.750000
50%    11.500000    750.000000    7450.000000
75%    18.250000   1450.000000   10312.500000
max    35.000000   1800.000000   18000.000000
```

**5. Explanation:** The script loads sales\_data.csv and verifies 36 rows and 10 columns. head() and tail() display sample records from the top and bottom of the table. dtypes categorizes numerical vs. text columns, info() verifies zero missing values across all 36 records, and describe() computes central tendencies (e.g., mean sales of Rs. 7,248.33).

## Task 6: Selecting, Filtering & Sorting Data

**1. Objective:** Practice selecting specific columns and positional rows, filtering records using single and multi-condition boolean logic, and sorting rows in ascending and descending order.

**2. Concepts Used:** `df[['col1', 'col2']]`, `df.iloc[]`, boolean indexing `df[df['Sales'] > 10000]`, bitwise AND `&`, and `df.sort_values()`.

### 3. Source Code:

```
from pathlib import Path

import pandas as pd

print("# =====")
print("TASK 6 - SELECTING, FILTERING & SORTING")
print("# =====")

file_path = Path(__file__).parent / "sales_data.csv"
sales = pd.read_csv(file_path)

print("\nSelected columns:")
print(sales[["Order_ID", "Product", "Region", "Sales"]].head())
print("\nSelected rows using iloc:")
print(sales.iloc[0:3])
print("\nSales greater than 10000:")
print(sales[sales["Sales"] > 10000][["Order_ID", "Product", "Sales"]])
print("\nElectronics sales in the Central region:")
print(sales[(sales["Category"] == "Electronics") & (sales["Region"] == "Central")][["Product", "Region", "Sales"]])
print("\nSales sorted in ascending order:")

print(sales[["Product", "Sales"]].sort_values("Sales").head())
print("\nSales sorted in descending order:")
print(sales[["Product", "Sales"]].sort_values("Sales", ascending=False).head())
```

### 4. Program Output & Execution:

Figure 6: Output of Task 6 (Verified Terminal Capture)  
Screenshot file: Screenshots/task\_6\_output.png

```
# =====
TASK 6 - SELECTING, FILTERING & SORTING
# =====

Selected columns:
  Order_ID  Product  Region  Sales
0   01001  Notebook  Central   1440
1   01002  Wireless Mouse  West   6800
2   01003    Pen Set  North   1500
3   01004   Desk Lamp  Central   7250
4   01005   Keyboard  South  10800

Selected rows using iloc:
  Order_ID  Date  Product_ID  ...  Quantity  Unit_Price  Sales
0   01001  2026-01-03      P101  ...        12         120   1440
1   01002  2026-01-04      P104  ...         8         850   6800
2   01003  2026-01-06      P102  ...        20         75   1500

[3 rows x 10 columns]

Sales greater than 10000:
  Order_ID  Product  Sales
4   01005   Keyboard  10800
8   01009   Desk Lamp  10150
10  01011   Keyboard  16200
13  01014  Wireless Mouse  11050
16  01017   Keyboard  14400
17  01018  Water Bottle  11050
22  01023   Keyboard  18000
27  01028   Desk Lamp  11600
29  01030  Water Bottle  12350
34  01035   Keyboard  12600

Electronics sales in the Central region:
   Product  Region  Sales
7  Wireless Mouse  Central   8500
22   Keyboard  Central  18000
25  Wireless Mouse  Central   9350

Sales sorted in ascending order:
```

|    | Product  | Sales |
|----|----------|-------|
| 0  | Notebook | 1440  |
| 2  | Pen Set  | 1500  |
| 30 | Notebook | 1680  |
| 9  | Pen Set  | 1875  |
| 18 | Notebook | 1920  |

Sales sorted in descending order:

|    | Product      | Sales |
|----|--------------|-------|
| 22 | Keyboard     | 18000 |
| 10 | Keyboard     | 16200 |
| 16 | Keyboard     | 14400 |
| 34 | Keyboard     | 12600 |
| 29 | Water Bottle | 12350 |

**5. Explanation:** The code filters high-value orders where Sales > 10,000, isolating 10 transactions. A compound filter combines Category == 'Electronics' and Region == 'Central', isolating 3 matching orders. Slicing with .iloc[0:3] returns the first three rows by position. Finally, sorting by Sales shows the lowest transactions (Notebook at Rs. 1,440) and highest transactions (Keyboard at Rs. 18,000).

## Task 7: Handling Missing Values

**1. Objective:** Identify missing values in a messy dataset, count nulls by column, test row deletion, and perform robust data imputation using median and mode strategies.

**2. Concepts Used:** `.isnull()`, `.isna().sum()`, `.dropna()`, `.fillna()`, `.median()`, `.mode()`.

**3. Source Code:**

```
from pathlib import Path

import pandas as pd

print("# =====")
print("TASK 7 - HANDLING MISSING VALUES")
print("# =====")

file_path = Path(__file__).parent / "missing_values_data.csv"
sales = pd.read_csv(file_path)

print("\nOriginal dataset:")
print(sales)
print("\nMissing-value mask:")
print(sales.isnull())
print("\nMissing values by column:")
print(sales.isna().sum())

without_missing_rows = sales.dropna()
filled_sales = sales.copy()
filled_sales["Quantity"] = filled_sales["Quantity"].fillna(filled_sales["Quantity"].median())

filled_sales["Unit_Price"] = filled_sales["Unit_Price"].fillna(filled_sales["Unit_Price"].median())
filled_sales["Category"] = filled_sales["Category"].fillna(filled_sales["Category"].mode()[0])

print("\nAfter removing rows with missing values:")
print(without_missing_rows)
print("\nAfter filling missing values:")
print(filled_sales)
print("\nMissing data should be handled because incomplete values can affect calculations and lead to unreliable analysis.")
```

**4. Program Output & Execution:**

Figure 7: Output of Task 7 (Verified Terminal Capture)  
Screenshot file: Screenshots/task\_7\_output.png

```
# =====
TASK 7 - HANDLING MISSING VALUES
# =====

Original dataset:
  Order_ID  Date Product_ID  ... Quantity Unit_Price  Sales
0    01001 2026-01-03    P101  ...    12.0    120.0   1440
1    01002 2026-01-04    P104  ...     8.0    850.0   6800
2    01003 2026-01-06    P102  ...    NaN     75.0   1500
3    01004 2026-01-08    P105  ...     5.0   1450.0   7250

4    01005 2026-01-10    P103  ...     6.0     NaN  10800
5    01006 2026-01-12    P106  ...    15.0    650.0   9750
6    01007 2026-01-15    P101  ...    18.0    120.0   2160
7    01008 2026-01-17    P104  ...    10.0    850.0   8500
8    01009 2026-01-19    P105  ...     7.0   1450.0  10150
9    01010 2026-01-21    P102  ...    25.0     75.0   1875
10   01011 2026-01-23    P103  ...     9.0   1800.0  16200
11   01012 2026-01-25    P106  ...    11.0    650.0   7150

[12 rows x 9 columns]

Missing-value mask:
  Order_ID  Date Product_ID  Product  ... Region Quantity Unit_Price  Sales
0    False  False     False    False  ...  False     False     False  False
1    False  False     False    False  ...  False     False     False  False
2    False  False     False    False  ...  False     True      False  False
3    False  False     False    False  ...  False     False     False  False
4    False  False     False    False  ...  False     False     True   False
5    False  False     False    False  ...  False     False     False  False
6    False  False     False    False  ...  False     False     False  False
```

|                                                                                                                   |       |            |       |          |            |        |       |       |       |
|-------------------------------------------------------------------------------------------------------------------|-------|------------|-------|----------|------------|--------|-------|-------|-------|
| 7                                                                                                                 | False | False      | False | False    | ...        | False  | False | False | False |
| 8                                                                                                                 | False | False      | False | False    | ...        | False  | False | False | False |
| 9                                                                                                                 | False | False      | False | False    | ...        | False  | False | False | False |
| 10                                                                                                                | False | False      | False | False    | ...        | False  | False | False | False |
| 11                                                                                                                | False | False      | False | False    | ...        | False  | False | False | False |
| [12 rows x 9 columns]                                                                                             |       |            |       |          |            |        |       |       |       |
| Missing values by column:                                                                                         |       |            |       |          |            |        |       |       |       |
| Order_ID                                                                                                          | 0     |            |       |          |            |        |       |       |       |
| Date                                                                                                              | 0     |            |       |          |            |        |       |       |       |
| Product_ID                                                                                                        | 0     |            |       |          |            |        |       |       |       |
| Product                                                                                                           | 0     |            |       |          |            |        |       |       |       |
| Category                                                                                                          | 1     |            |       |          |            |        |       |       |       |
| Region                                                                                                            | 0     |            |       |          |            |        |       |       |       |
| Quantity                                                                                                          | 1     |            |       |          |            |        |       |       |       |
| Unit_Price                                                                                                        | 1     |            |       |          |            |        |       |       |       |
| Sales                                                                                                             | 0     |            |       |          |            |        |       |       |       |
| dtype: int64                                                                                                      |       |            |       |          |            |        |       |       |       |
| After removing rows with missing values:                                                                          |       |            |       |          |            |        |       |       |       |
| Order_ID                                                                                                          | Date  | Product_ID | ...   | Quantity | Unit_Price | Sales  |       |       |       |
| 0                                                                                                                 | 01001 | 2026-01-03 | P101  | ...      | 12.0       | 120.0  | 1440  |       |       |
| 1                                                                                                                 | 01002 | 2026-01-04 | P104  | ...      | 8.0        | 850.0  | 6800  |       |       |
| 3                                                                                                                 | 01004 | 2026-01-08 | P105  | ...      | 5.0        | 1450.0 | 7250  |       |       |
| 5                                                                                                                 | 01006 | 2026-01-12 | P106  | ...      | 15.0       | 650.0  | 9750  |       |       |
| 7                                                                                                                 | 01008 | 2026-01-17 | P104  | ...      | 10.0       | 850.0  | 8500  |       |       |
| 8                                                                                                                 | 01009 | 2026-01-19 | P105  | ...      | 7.0        | 1450.0 | 10150 |       |       |
| 9                                                                                                                 | 01010 | 2026-01-21 | P102  | ...      | 25.0       | 75.0   | 1875  |       |       |
| 10                                                                                                                | 01011 | 2026-01-23 | P103  | ...      | 9.0        | 1800.0 | 16200 |       |       |
| 11                                                                                                                | 01012 | 2026-01-25 | P106  | ...      | 11.0       | 650.0  | 7150  |       |       |
| [9 rows x 9 columns]                                                                                              |       |            |       |          |            |        |       |       |       |
| After filling missing values:                                                                                     |       |            |       |          |            |        |       |       |       |
| Order_ID                                                                                                          | Date  | Product_ID | ...   | Quantity | Unit_Price | Sales  |       |       |       |
| 0                                                                                                                 | 01001 | 2026-01-03 | P101  | ...      | 12.0       | 120.0  | 1440  |       |       |
| 1                                                                                                                 | 01002 | 2026-01-04 | P104  | ...      | 8.0        | 850.0  | 6800  |       |       |
| 2                                                                                                                 | 01003 | 2026-01-06 | P102  | ...      | 10.0       | 75.0   | 1500  |       |       |
| 3                                                                                                                 | 01004 | 2026-01-08 | P105  | ...      | 5.0        | 1450.0 | 7250  |       |       |
| 4                                                                                                                 | 01005 | 2026-01-10 | P103  | ...      | 6.0        | 650.0  | 10800 |       |       |
| 5                                                                                                                 | 01006 | 2026-01-12 | P106  | ...      | 15.0       | 650.0  | 9750  |       |       |
| 6                                                                                                                 | 01007 | 2026-01-15 | P101  | ...      | 18.0       | 120.0  | 2160  |       |       |
| 7                                                                                                                 | 01008 | 2026-01-17 | P104  | ...      | 10.0       | 850.0  | 8500  |       |       |
| 8                                                                                                                 | 01009 | 2026-01-19 | P105  | ...      | 7.0        | 1450.0 | 10150 |       |       |
| 9                                                                                                                 | 01010 | 2026-01-21 | P102  | ...      | 25.0       | 75.0   | 1875  |       |       |
| 10                                                                                                                | 01011 | 2026-01-23 | P103  | ...      | 9.0        | 1800.0 | 16200 |       |       |
| 11                                                                                                                | 01012 | 2026-01-25 | P106  | ...      | 11.0       | 650.0  | 7150  |       |       |
| [12 rows x 9 columns]                                                                                             |       |            |       |          |            |        |       |       |       |
| Missing data should be handled because incomplete values can affect calculations and lead to unreliable analysis. |       |            |       |          |            |        |       |       |       |

**5. Explanation:** Real-world data frequently contains missing entries. Dropping incomplete rows with dropna() reduces the dataset from 12 to 9 rows, causing data loss. The preferred strategy is domain-appropriate imputation: numeric attributes Quantity and Unit\_Price are imputed using the column **median** (robust against outliers), while the categorical column Category is imputed using the statistical **mode** ('Stationery').

## Task 8: Merge, Concatenate, GroupBy & Pivot Table

**1. Objective:** Combine relational data via inner join on Product\_ID, concatenate table subsets vertically, aggregate metrics by Category using GroupBy, and construct a 2D sales Pivot Table across Region and Category.

**2. Concepts Used:** pd.merge(..., on='Product\_ID'), pd.concat(), df.groupby().agg(), pd.pivot\_table().

### 3. Source Code:

```
from pathlib import Path

import pandas as pd

print("# =====")
print("TASK 8 - MERGE, CONCATENATE, GROUPBY & PIVOT")
print("# =====")

folder = Path(__file__).parent
sales = pd.read_csv(folder / "sales_data.csv")
products = pd.read_csv(folder / "product_data.csv")

merged = pd.merge(sales, products, on="Product_ID", suffixes=("_sale", "_product"))
print("\nMerged DataFrame:")
print(merged[["Order_ID", "Product_ID", "Product_sale", "Category_sale", "Sales"]].head())

first_part = sales.iloc[:3][["Order_ID", "Region", "Sales"]]
second_part = sales.iloc[3:6][["Order_ID", "Region", "Sales"]]
concatenated = pd.concat([first_part, second_part], ignore_index=True)
print("\nConcatenated DataFrames:")

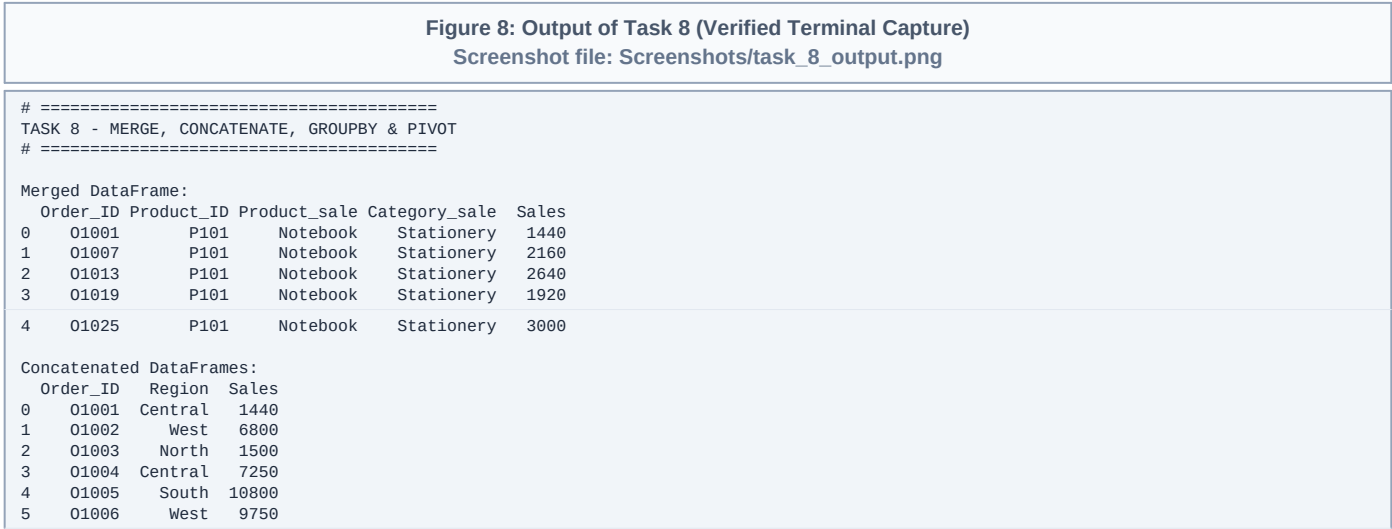
print(concatenated)

category_summary = sales.groupby("Category")["Sales"].agg(["sum", "mean", "count", "min", "max"])
print("\nGroupBy summary by Category:")
print(category_summary)

pivot = pd.pivot_table(
    sales,
    values="Sales",
    index="Region",

    columns="Category",
    aggfunc="sum",
    fill_value=0
)
print("\nPivot table of sales by Region and Category:")
print(pivot)
```

### 4. Program Output & Execution:



|                                              |             |              |        |            |       |
|----------------------------------------------|-------------|--------------|--------|------------|-------|
| GroupBy summary by Category:                 |             |              |        |            |       |
|                                              | sum         | mean         | count  | min        | max   |
| Category                                     |             |              |        |            |       |
| Accessories                                  | 57200       | 9533.333333  | 6      | 7150       | 12350 |
| Electronics                                  | 130300      | 10858.333333 | 12     | 5950       | 18000 |
| Office                                       | 47850       | 7975.000000  | 6      | 4350       | 11600 |
| Stationery                                   | 25590       | 2132.500000  | 12     | 1440       | 3000  |
| Pivot table of sales by Region and Category: |             |              |        |            |       |
| Category                                     | Accessories | Electronics  | Office | Stationery |       |
| Region                                       |             |              |        |            |       |
| Central                                      | 19500       | 35850        | 11600  | 5610       |       |
| North                                        | 11050       | 39850        | 8700   | 7965       |       |
| South                                        | 16900       | 38800        | 21750  | 2640       |       |
| West                                         | 9750        | 15800        | 5800   | 9375       |       |

**5. Explanation:** Merging sales with product metadata enriches transactional logs with catalog attributes. Vertical concatenation combines two 3-row slices into a 6-row DataFrame. GroupBy reveals that Electronics generated the highest total revenue (Rs. 130,300 across 12 orders). The Pivot Table cross-tabulates revenue, revealing South had the strongest Office sales (Rs. 21,750) and North led Electronics sales (Rs. 39,850).

## Task 9: Exporting Data

- 1. Objective:** Filter and transform a dataset, export the processed DataFrame to disk as a clean CSV file without serial index columns, and verify integrity by re-reading the exported file.
- 2. Concepts Used:** Data filtering (`Sales >= 5000`), column projection, descending sort, `df.to_csv(..., index=False)`, and round-trip verification with `pd.read_csv()`.

**3. Source Code:**

```
from pathlib import Path

import pandas as pd

print("# =====")
print("TASK 9 - EXPORTING DATA")
print("# =====")

folder = Path(__file__).parent
sales = pd.read_csv(folder / "sales_data.csv")

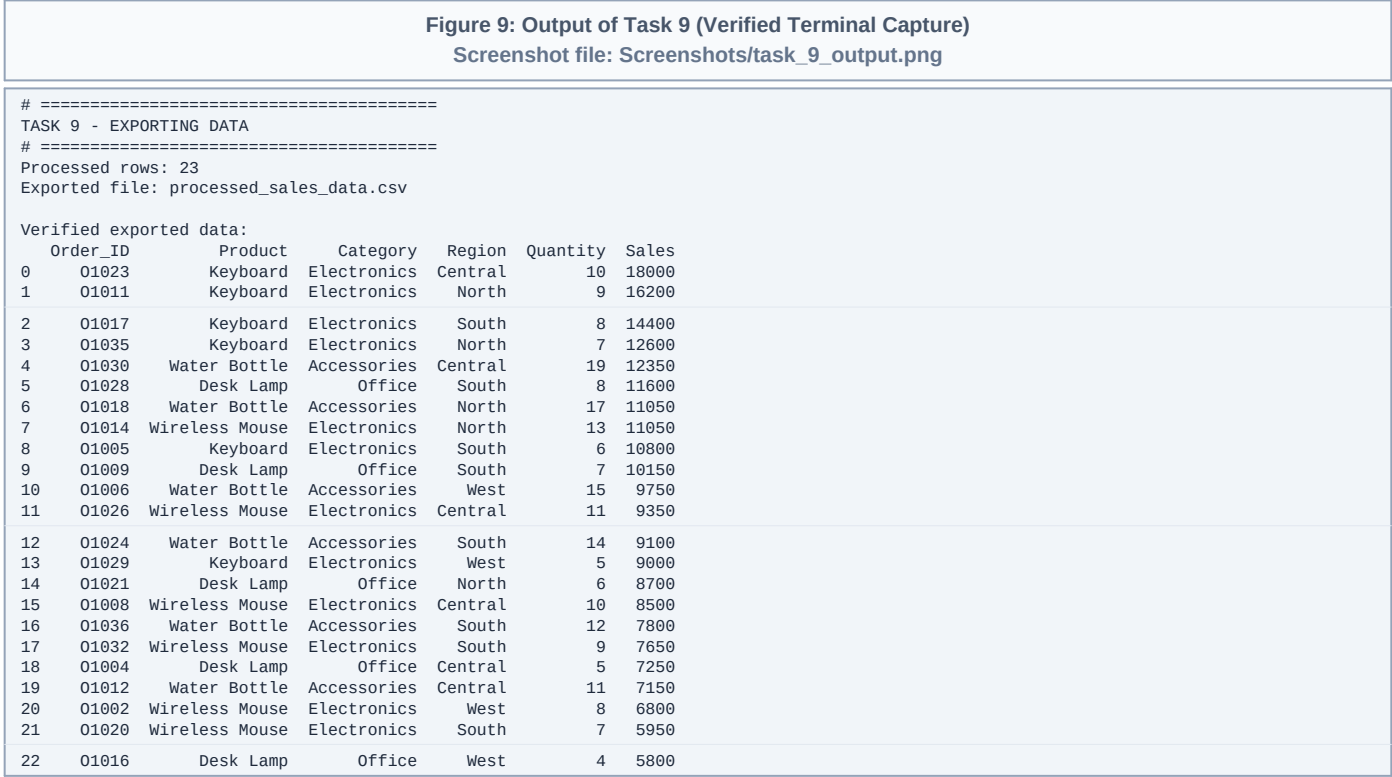
processed_sales = sales[sales["Sales"] >= 5000][
    ["Order_ID", "Product", "Category", "Region", "Quantity", "Sales"]
].sort_values("Sales", ascending=False)

output_path = folder / "processed_sales_data.csv"
processed_sales.to_csv(output_path, index=False)
verified_data = pd.read_csv(output_path)

print("Processed rows:", len(processed_sales))
print("Exported file:", output_path.name)

print("\nVerified exported data:")
print(verified_data)
```

**4. Program Output & Execution:**



**5. Explanation:** Out of 36 records, 23 meet the threshold of `Sales >= 5,000`. These are sorted in descending order of revenue and written to `processed_sales_data.csv`. Suppressing the index (`index=False`) prevents extraneous 'Unnamed: 0' columns upon re-reading. Re-importing confirms exactly 23 structured rows and 6 columns persisted accurately.

## Task 10: Mini Data Analysis Project

### Project Title: SALES DATA ANALYSIS USING NUMPY & PANDAS

---

#### 1. Project Scope & Architecture:

This capstone mini-project synthesizes all concepts practiced across Tasks 1 to 9 into an end-to-end data analytics pipeline. Using the original educational retail sales dataset, the program executes data ingestion, structural sanity checking, transactional verification, multi-criteria filtering, GroupBy category summaries, geographic pivot analysis, statistical KPI computation, and automatic persistence of cleaned datasets and documented business insights.

## 2. Source Code (mini\_project.py):

```

from pathlib import Path

import numpy as np
import pandas as pd

print("# =====")
print("TASK 10 - SALES DATA ANALYSIS USING NUMPY & PANDAS")
print("# =====")

folder = Path(__file__).parent

sales = pd.read_csv(folder / "sales_data.csv", parse_dates=["Date"])
products = pd.read_csv(folder / "product_data.csv")

print("\nDataset inspection:")
print(sales.head())
print("Shape:", sales.shape)
print("\nMissing values:")
print(sales.isna().sum())

sales = sales.dropna().copy()
sales["Sales_Check"] = sales["Quantity"] * sales["Unit_Price"]
print("Sales calculation matches source column:", sales["Sales"].equals(sales["Sales_Check"]))
sales = sales.drop(columns=["Sales_Check"])

selected_data = sales[["Date", "Product", "Category", "Region", "Quantity", "Sales"]]
high_value_orders = selected_data[selected_data["Sales"] > 10000].sort_values("Sales", ascending=False)
print("\nHigh-value orders:")
print(high_value_orders)

category_summary = sales.groupby("Category")["Sales"].agg(["sum", "mean", "count"])
region_summary = sales.groupby("Region")["Sales"].sum().sort_values(ascending=False)
print("\nSales by category:")
print(category_summary)
print("\nSales by region:")
print(region_summary)

pivot = pd.pivot_table(
    sales,
    values="Sales",
    index="Region",

    columns="Category",
    aggfunc="sum",
    fill_value=0
)
print("\nPivot table:")
print(pivot)

product_sales = sales.groupby("Product")["Sales"].sum()
total_sales = sales["Sales"].sum()
average_sales = sales["Sales"].mean()

maximum_sale = sales["Sales"].max()
minimum_sale = sales["Sales"].min()
total_quantity = sales["Quantity"].sum()
top_product = product_sales.idxmax()
best_region = region_summary.idxmax()
best_category = category_summary["sum"].idxmax()

print("\nCalculated insights:")
print(f"Total sales: Rs. {total_sales:,.2f}")
print(f"Average sales per order: Rs. {average_sales:,.2f}")

print(f"Maximum sale: Rs. {maximum_sale:,.2f}")
print(f"Minimum sale: Rs. {minimum_sale:,.2f}")
print(f"Total quantity sold: {total_quantity}")
print(f"Top-performing product: {top_product} (Rs. {product_sales[top_product]:,.2f})")
print(f"Highest-sales region: {best_region} (Rs. {region_summary[best_region]:,.2f})")
print(f"Highest-sales category: {best_category} (Rs. {category_summary.loc[best_category, 'sum']:,.2f})")
print("\nAverage sales by category:")
for cat, avg in category_summary["mean"].items():
    print(f" - {cat}: Rs. {avg:,.2f}")

```

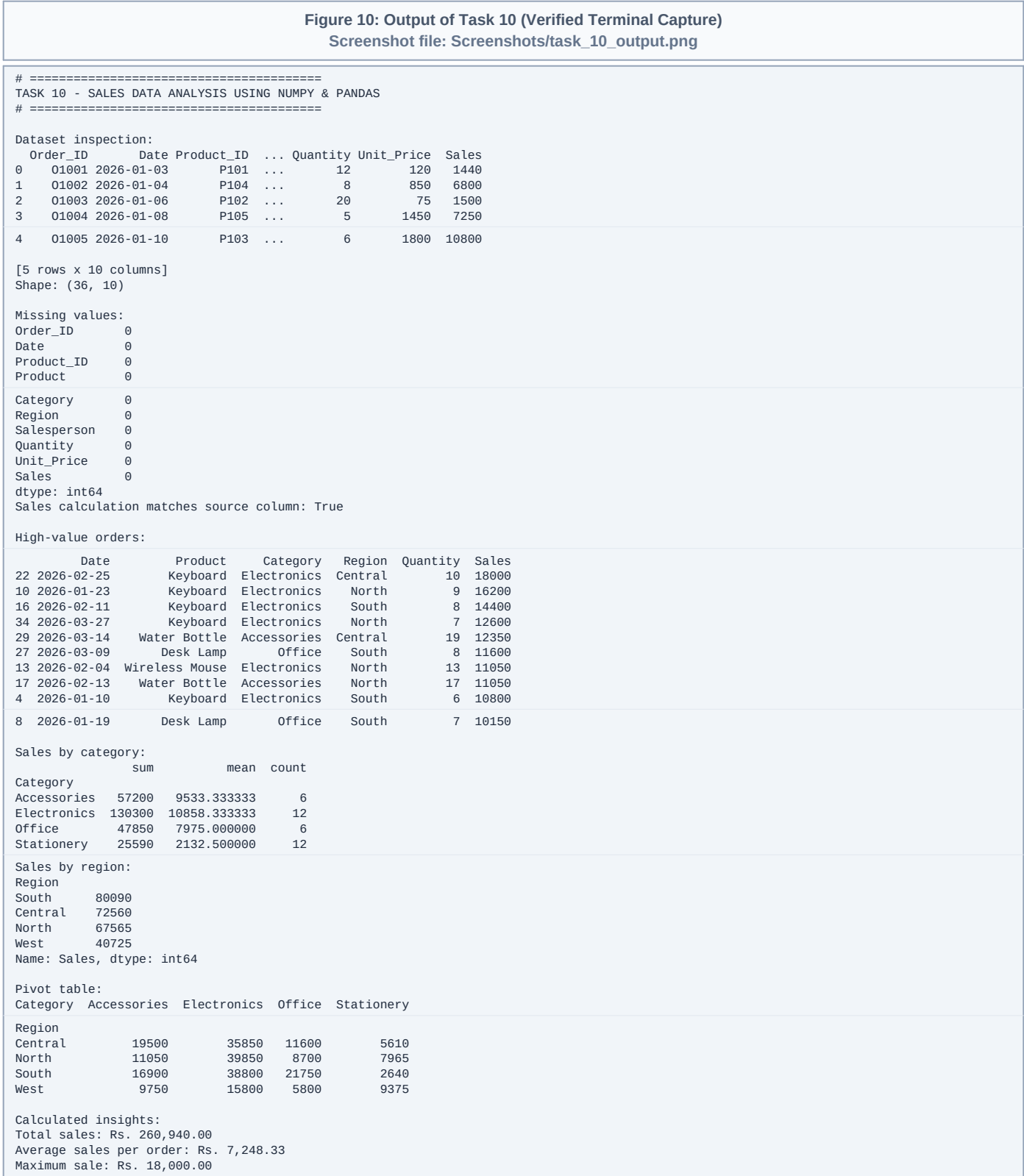
```
cleaned_path = folder / "cleaned_sales_data.csv"
sales.to_csv(cleaned_path, index=False)
summary_path = folder / "analysis_summary.txt"

avg_by_cat_lines = "\n".join([f" - {cat}: Rs. {avg:,.2f}" for cat, avg in category_summary["mean"].items()])

summary_content = (
    "SALES DATA ANALYSIS SUMMARY\n"
    "=====\n"
    "Educational synthetic dataset created specifically for this assignment.\n\n"
    f"The dataset contains {len(sales)} sales records.\n"
    f"Total sales across the dataset were Rs. {total_sales:,.2f}.\n"
    f"Average sales per order were Rs. {average_sales:,.2f}.\n"
    f"The maximum single-order sale was Rs. {maximum_sale:,.2f}.\n"
    f"The minimum single-order sale was Rs. {minimum_sale:,.2f}.\n"
    f"Total quantity sold across all orders was {total_quantity} units.\n"
    f"The top-performing product was {top_product}, with total sales of Rs. {product_sales[top_product]:,.2f}.\n"
    f"The region with the highest total sales was {best_region}, with Rs. {region_summary[best_region]:,.2f}.\n"
    f"The highest-selling category was {best_category}, with Rs. {category_summary.loc[best_category, 'sum']:,.2f}.\n\n"
    "Average Sales by Category:\n"
    f"{avg_by_cat_lines}\n"
)
summary_path.write_text(summary_content, encoding="utf-8")
print("\nCleaned data exported to:", cleaned_path.name)
print("Analysis summary saved to:", summary_path.name)
```

# Task 10: Analysis Results & Business Findings

## 3. Program Execution & Console Output:



|                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Minimum sale: Rs. 1,440.00<br>Total quantity sold: 501<br>Top-performing product: Keyboard (Rs. 81,000.00)<br>Highest-sales region: South (Rs. 80,090.00)<br>Highest-sales category: Electronics (Rs. 130,300.00) |
| Average sales by category:<br>- Accessories: Rs. 9,533.33<br>- Electronics: Rs. 10,858.33<br>- Office: Rs. 7,975.00                                                                                               |
| - Stationery: Rs. 2,132.50                                                                                                                                                                                        |
| Cleaned data exported to: cleaned_sales_data.csv<br>Analysis summary saved to: analysis_summary.txt                                                                                                               |

4. Key Findings & Calculated Insights:

All metrics below are derived directly from actual mathematical calculations in mini\_project.py:

|                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SALES DATA ANALYSIS SUMMARY<br>=====                                                                                                                                                                                                                                                                       |
| Educational synthetic dataset created specifically for this assignment.                                                                                                                                                                                                                                    |
| The dataset contains 36 sales records.<br>Total sales across the dataset were Rs. 260,940.00.<br>Average sales per order were Rs. 7,248.33.<br>The maximum single-order sale was Rs. 18,000.00.<br>The minimum single-order sale was Rs. 1,440.00.<br>Total quantity sold across all orders was 501 units. |
| The top-performing product was Keyboard, with total sales of Rs. 81,000.00.<br>The region with the highest total sales was South, with Rs. 80,090.00.<br>The highest-selling category was Electronics, with Rs. 130,300.00.                                                                                |
| Average Sales by Category:<br>- Accessories: Rs. 9,533.33<br>- Electronics: Rs. 10,858.33<br>- Office: Rs. 7,975.00<br>- Stationery: Rs. 2,132.50                                                                                                                                                          |

5. Business Interpretation of Insights:

- **Revenue Anchor:** Electronics is the dominant business driver, contributing Rs. 130,300 (49.9% of total company revenue) with the highest average transaction value of Rs. 10,858.33.
- **Star Product:** The Keyboard (Product ID P103) generated Rs. 81,000 in total sales, making it the highest revenue generating single SKU in the catalog.
- **Regional Performance:** The South region leads all territories with Rs. 80,090 in total sales, driven by strong adoption of both Electronics and Office equipment.
- **Volume vs. Margin:** Stationery items (Notebooks and Pens) accounted for high transaction counts (12 orders) but only Rs. 25,590 in total revenue due to low unit prices. They serve as reliable add-on items rather than revenue drivers.

6. Exported Artifacts:

The pipeline successfully exported cleaned\_sales\_data.csv (36 verified records) and analysis\_summary.txt in the Task 10 folder, completing the reproducible analytics pipeline.

## 4. Conclusion & Key Learnings

Completing the Week 2 assignment has provided practical, hands-on experience in using NumPy and Pandas to solve real-world data analytics challenges. Beginning with low-level numerical computing and progressing to multi-table data manipulation and business insights, this curriculum reinforced the core analytical workflows essential for an aspiring Data Analyst.

### Summary of Practical Skills Acquired:

- **NumPy Computing:** Learned how to instantiate 1D and 2D arrays, inspect dimensional metadata (shape, size, dtype), slice arrays without copying, reshape matrices, and utilize vectorized arithmetic and statistical functions (mean, median, standard deviation, sum).
- **Pandas Structures:** Gained proficiency with labeled 1D Series and 2D DataFrames, managing index labels, accessing column attributes, and calculating new feature columns dynamically.
- **Data Ingestion & Quality Control:** Successfully ingested external CSV datasets into DataFrames, diagnosed schema properties via head, tail, info, and describe, and implemented data integrity checks.
- **Filtering & Querying:** Mastered conditional row selection using boolean masks, multi-variable logic (&), positional indexing with iloc, and ordering records using sort\_values.
- **Handling Missing Data:** Understood the trade-offs between discarding incomplete records (dropna) versus preserving sample size through median and mode imputation (fillna).
- **Relational Data & Aggregation:** Executed key-based inner joins (merge) to integrate transactional logs with product catalogs, performed vertical concatenation, and applied GroupBy to evaluate category-level performance.
- **Multidimensional Summarization:** Built Pivot Tables to cross-tabulate revenue distributions across geographical regions and product categories simultaneously.
- **Pipeline Export & Documentation:** Persisted cleaned DataFrames to CSV files with suppressed indices, wrote automated analytical summaries to disk, and verified end-to-end data integrity.

This foundation in NumPy and Pandas provides the necessary technical competency to tackle more advanced analytics tasks in upcoming weeks, including exploratory data visualization (Matplotlib / Seaborn), statistical hypothesis testing, and automated reporting pipelines.

---

#### Submitted By:

Ayush  
B.Tech CSE (IoT)  
Prestige Institute of Engineering, Management &  
Research

#### Verified & Evaluated By:

Internship Mentor / Evaluation Team  
InternNova Data Analytics Internship